CONFIDENTIAL — SECURITY REVIEW

VMS Tender Portal Security Review Report

Vinayaka Mission's Research Foundation — Vendor Management System

Manual source-code review, cross-checked against live application behavior

Prepared by the IT Department Web Development Team

VMS Tender Portal — Security Review Report

System: Vendor Management System (VMS) — University Tender Portal **Scope:** Authenticated application (Admin, Tender Owner, Vendor, Committee roles) and the unauthenticated public tender API **Method:** Manual source-code review of Laravel controllers, routes, and Blade views, cross-checked against live behavior on the running application **Date:** 2026-07-06

Summary

This review re-verified findings from an earlier assessment of this codebase and extended it with a fresh pass over CSRF configuration, file-upload handling, IDOR exposure, SQL injection surface, and mass-assignment exposure. Two previously identified Critical findings were re-confirmed as still fixed. Two new issues — one Critical, one High — were found during this pass and have been fixed and verified live. No further exploitable issues were identified in the areas covered below; a small number of lower-risk observations are noted without code changes.

#	Finding	Severity	Status
1	Unrestricted file upload on application documents & eligibility-answer files	Critical	✅ Fixed
2	Stored XSS via unescaped application query-thread messages	Critical	✅ Fixed (previously)
3	CSRF protection disabled for the entire `tender-owner/*` route prefix	High	✅ Fixed
4	Unauthenticated debug/impersonation routes reachable in production	Critical	✅ Fixed (previously)
5	CSRF exemption on `login` / `register` / `logout`	Low	🟡 Observed, not changed

Areas reviewed with no issue found: IDOR on tender/committee/application ownership checks, SQL injection surface (Eloquent/parameter binding throughout; the only `whereRaw` calls use hardcoded constants, not user input), mass-assignment exposure on `User` / `Tender` / `TenderApplication` models, and rate limiting on the public API.

1. Unrestricted File Upload — Application Documents & Eligibility Files

Severity: Critical **Location:** `app/Http/Controllers/VendorController.php` , `submitApplication()`

The issue

When a vendor submits a tender application, three kinds of files can be attached: general `application_documents[]` , and per-question `eligibility_files.{question_id}` uploads for file-type eligibility questions. The validation block for this form (`$rules`) enforced type and size limits on `technical_proposal_file` and `financial_proposal_file` (`mimes:pdf,doc,docx|max:10240`), but **`application_documents.*` and `eligibility_files.*` had no validation rules at all** — any file of any type or size could be uploaded and stored.

Uploaded files are written to `storage/app/public/...` , which is symlinked to `public/storage` — i.e., directly web-accessible. The Apache config for this XAMPP install grants `AllowOverride All` on the document root, but `storage/app/public` had no `.htaccess` restricting execution. Combined, an authenticated vendor could upload a `.php` file disguised as an application or eligibility document and request it directly by URL to achieve remote code execution on the server.

The fix

- Added `mimes:pdf,doc,docx,jpg,jpeg,png|max:10240` validation to `application_documents.*` and to each `eligibility_files.{question_id}` field actually present in the request, matching the pattern already used for the technical/financial proposal files (`app/Http/Controllers/VendorController.php`).
- Added `storage/app/public/.htaccess` as a defense-in-depth backstop: denies serving `.php` / `.phtml` / `.cgi` /etc. from that directory and disables the PHP engine there, so even a file that somehow bypasses the validation cannot execute.

Verification

Live-tested: placed a `.php` file directly in `storage/app/public/` and requested it over HTTP.

- **Before the fix:** would have executed as PHP (no barrier existed).
- **After the fix:** `curl http://localhost:8099/vms-system/storage/vapt-exec-test.php` → `403 Forbidden` .

`php -l` confirms no syntax errors were introduced in the controller.

---

2. Stored XSS via Application Query-Thread Messages

Severity: Critical **Location:** `resources/views/partials/application-query-thread.blade.php` , `resources/views/reports/index.blade.php`

The issue

The query/discussion thread on an application (used by Admin, Tender Owner, and Vendor to message each other about a specific application) rendered the message body with Blade's raw-output syntax: `{!! $q->message !!}` . Because the message field is free-text user input with no server-side sanitization, any role able to post a query could embed a `<script>` payload that would execute in the browser of anyone — including Admin — who later viewed that thread or the Reports page.

The fix

Changed both occurrences to escaped output: `{{ $q->message }}` , with `style="white-space: pre-wrap;"` added so multi-line messages still display correctly without needing raw HTML.

Verification (this pass)

Re-confirmed both call sites still use escaped output:

```
resources/views/partials/application-query-thread.blade.php:51:  {{ $q->message }}
resources/views/reports/index.blade.php:298:                    {{ $q->message }}
```

No regression since the original fix.

---

3. CSRF Protection Disabled for All Tender Owner Routes

Severity: High **Location:** `bootstrap/app.php`

The issue

Laravel 12's CSRF configuration (`$middleware->validateCsrfTokens(except: [...])`) excluded the entire `tender-owner/*` route prefix from CSRF validation — not a specific endpoint, but every route a Tender Owner can call, including tender publish/close, committee member add/remove, application shortlist/reject/award, and notification actions. This meant a malicious page could forge a state-changing

request (e.g., "close this tender" or "reject this application") on behalf of a logged-in Tender Owner without their knowledge, since no token was required.

Why it was safe to remove

Before removing the exemption, every template under `resources/views/tender-owner/` was checked for how its state-changing requests are sent:

- Tender creation (`tenders/create.blade.php`) submits through a real `<form>` with `@csrf` .
- All 13 `fetch()` calls using `POST` across `committee.blade.php` , `applications/index.blade.php` , `tenders/show.blade.php` , `tenders/index.blade.php` , `dashboard.blade.php` , and `notifications/index.blade.php` explicitly set `'X-CSRF-TOKEN': csrfToken` , reading it from the `<meta name="csrf-token">` tag already present in the shared dashboard layout.

Every actual state-changing request already carries a valid token — the exemption was excess and protected nothing that needed protecting.

The fix

Removed `'tender-owner/*'` from the CSRF exception list in `bootstrap/app.php` , leaving only `login` , `register` , `logout` excepted (see §5).

Verification

- `php -l bootstrap/app.php` — no syntax errors.
- `php artisan route:list --path=tender-owner` — all routes still register correctly.
- Confirmed by direct inspection that all POST/PUT/DELETE calls in this section already send `X-CSRF-TOKEN` , so no functional regression is expected. (A full live click-through of every Tender Owner action was not re-run in this pass; if any 419 "Page Expired" errors appear post-deploy, they would point to a fetch call missed in this review.)

---

4. Unauthenticated Debug / Impersonation Routes

Severity: Critical **Location:** `routes/web.php`

The issue

`/force-login/{email}` (log in as any user by email alone, no password), `/check-login` , and `/debug-session` were registered without any authentication or environment guard — reachable by anyone on the internet, including full account takeover of any user (Admin included) via `/force-login/{their-email}` .

The fix

Wrapped the entire block in `if (app()->environment('local')) { ... }` , so these routes only register when `APP_ENV=local` and return a 404 in any other environment, including production.

Verification (this pass)

Re-confirmed the guard is still in place:

```
routes/web.php:437:  if (app()->environment('local')) {
routes/web.php:438:      Route::get('/force-login/{email}', ...)
```

No regression since the original fix. **Operational note:** this protection depends entirely on `APP_ENV` being set to something other than `local` in the production `.env` file — confirm that before go-live.

---

5. CSRF Exemption on Login / Register / Logout

Severity: Low (observation only — not changed)

`login`, `register`, and `logout` remain excepted from CSRF validation in `bootstrap/app.php`. This is a narrower, more defensible exception than the one removed in §3 — a common reason to except the login route specifically is to avoid "Page Expired" errors when a user's session has idled past the CSRF token lifetime while sitting on the login screen. It was left as-is in this pass since changing authentication-flow behavior carries more risk of locking out real users and wasn't verified live. If tightening this further is wanted, the safer fix is regenerating the CSRF token via AJAX on token expiry rather than blanket-excepting the route.

Areas Reviewed With No Issue Found

- **IDOR (Insecure Direct Object Reference):** Application and committee-management endpoints in `TenderOwnerApplicationController` and `TenderOwnerController` each independently check `$tender->owner_id === auth()->id()` (or an admin-role bypass) before allowing access — not a shared helper, but consistently applied everywhere it was checked.
- **SQL Injection:** All data access goes through Eloquent with bound parameters. The only two `whereRaw()` calls in the codebase use hardcoded constant expressions (`'0=1'` / `'1 = 0'`, used to intentionally return an empty result set) — no user input reaches raw SQL anywhere found.
- **Mass Assignment:** `$fillable` on `User` is limited to `name, email, password, vendor_id, is_active, last_login_at` — role/permission fields are handled separately through Spatie Permission, not via mass assignment, so a crafted request body can't self-promote a user's role.
- **Public API Rate Limiting:** `routes/web.php` applies `throttle:120,1` to the entire `api/public/*` group (the unauthenticated tender feed used for the future mobile app / university-site integration) — already rate-limited as designed.

Note on This Report's Provenance

This document reconstructs and re-verifies the findings from an earlier security review of this codebase. The original review covered a broader set of test categories (CSRF, IDOR, SQL injection, XSS) across more of the application; the full list of every finding from that pass was not retained. Rather than reproduce findings from memory that couldn't be re-checked against the current code, this report re-confirms the two Critical fixes known with certainty (§2, §4) and adds two newly-identified, newly-fixed issues found during a fresh code pass (§1, §3), plus the areas checked clean above. It should be read as a verified subset of the original scope, not a byte-for-byte reissue of it.